

ENEL712 Exam Study Guide

Complete Reference — All Lectures

“Try to understand the role of each item or component in the overall embedded digital system design.”

— Dr. Al-Anbuky

ENEL712 · Embedded Digital Systems · Auckland University of Technology

Answer Framework

What the exam tests: A real embedded system scenario with questions about WHY each component is needed, what its constraints are, and how components interact. Not definitions — applied problem solving. The disk synchronization and vineyard IoT examples are the templates.

The one technique that earns the most marks: for any question, trace the data flow, then state the constraint:

Physical quantity → Sensor → MCU peripheral/register → Software → Output/Comms

Universal answer structure:

1. **Identify** the physical quantity (RPM, temperature, voltage) — sets up why everything exists.
2. **Trace the data flow** end-to-end. Naming this chain earns marks in almost every question.
3. **Justify** each component: not just “I²C” but “I²C because three sensors share two wires; SPI would need three chip-selects.”
4. **Calculate:** sample rate, SRAM needed, UART throughput, OCR value, battery life. Showing working earns full marks even if the final number is slightly off.
5. **Identify trade-offs:** “30 KB exceeds the 8 KB SRAM — must stream via UART.”
6. **Classify:** Functional = *what* it does; Non-Functional = *how well*.

Contents

1	The Two Exam Scenarios	1
1.1	How each component maps to each scenario	1
2	Lectures 1–2: MCU Architecture	2
2.1	What makes an MCU an MCU	2
2.2	Register file — why 32 general-purpose registers?	2
2.3	RISC characteristics (AVR)	3
2.4	The 2-stage pipeline	3
2.5	Harvard vs von Neumann	3
3	Lecture 3: Memory System	4
3.1	Memory overview	4
3.2	Memory decision table	4
3.3	SRAM buffer sizing	4
3.4	SRAM memory map	5
3.5	EEPROM endurance	5
3.6	Clocking system	5
4	Lecture 4: Advanced Hardware	6
4.1	ARM — why it exists	6
4.2	MCU vs MPU	7
4.3	DMA and event systems	7
5	Lecture 5: Timers, ADC, PWM, Interrupts, Communications	8
5.1	Timers — the backbone of real-time	8
5.2	ADC — sensing the physical world	8
5.3	PWM — controlling actuators	9
5.4	Interrupts — real-time response	10
5.5	UART / SPI / I ² C — protocol comparison	10
5.6	USB and Ethernet	11
6	Lecture 6: IoT Architecture and Power Management	12
6.1	IoT hierarchy	12
6.2	Power and sleep modes	12

7	Lecture 7: Hardware Design Process	14
7.1	The process order — why I/O first?	14
7.2	I/O device categories	14
7.3	Power supply: switching vs linear	14
7.4	MCU selection criteria (vineyard example)	14
8	Lecture 8: Software Engineering	16
8.1	Four process activities	16
8.2	Functional vs non-functional requirements	16
8.3	The project report checklist (Dr. Al-Anbuky's 11 points)	17
8.4	State machine modelling	17
8.5	Software testing	17
9	Lecture 9: Software Architectures and Defensive Design	18
9.1	Architecture patterns	18
9.2	Defensive software design	19
9.3	Hardware/software partitioning	19
10	Lecture 10: Modular Software Design	21
10.1	The three-level hierarchy	21
10.2	Driver level architecture	21
11	Fully Worked Examples	22
	Quick Reference	24

Chapter 1

The Two Exam Scenarios

Exam Scenario

Disk Synchronisation. Two slotted disks on perpendicular axes must pass through a shared zone without collision. Requires: tachogenerator → ADC for speed sensing; PWM for motor control; Input Capture timer for slot position; UART/USB for data logging; PID control; step-response analysis.

Exam Scenario

Vineyard IoT Weather Station. Remote environmental monitoring (temperature, humidity, rainfall, sunlight) in NZ vineyards, self-powered by battery/solar, wireless to farmhouse. Requires: I²C sensors; ADC; sleep modes; ZigBee via UART; EEPROM for settings; battery life calculation.

1.1 How each component maps to each scenario

Component	Disk Sync	Vineyard IoT
ADC	Tachogenerator speed voltage	Analog light/rainfall sensors
PWM	Motor speed control	Not primary
Timer CTC	ADC trigger at 200 Hz	WDT wakeup interval (60 s)
Timer Input Capture	Timestamp slot events	Not used
Interrupts	ADC complete, slot detection	WDT wakeup, RTC alarm
UART	Stream data to PC	ZigBee wireless module
I ² C	Not primary	Temperature, humidity, RTC
SRAM	Circular buffer (8 KB limit!)	Small sensor buffers
EEPROM	PID gains, motor calibration	ZigBee channel, calibration
Sleep modes	Not priority	Critical — months of battery
State machine	IDLE/RAMP/STEADY/STOP loop	SLEEP/SENSE/TX/SLEEP loop

Chapter 2

Lectures 1–2: MCU Architecture

AT90USB1287 Specs — Know These

Architecture: 8-bit AVR RISC, Harvard Clock: up to 16 MHz MIPS: 16 at 16 MHz
Flash: 128 KB SRAM: 8 KB EEPROM: 4 KB
ADC: 10-bit, 8 channels, max ≈ 9.6 kHz Timers: T0/T2 (8-bit), T1/T3 (16-bit)
Interfaces: SPI, UART, TWI (I²C), USB Sleep (Power-down): $< 1 \mu\text{A}$

2.1 What makes an MCU an MCU

An MCU integrates everything on one chip: CPU core, program memory (Flash), data memory (SRAM), non-volatile settings (EEPROM), and all I/O peripherals (ADC, timers, comms). The key advantage over an MPU: no external components, single 3.3 V/5 V supply, deterministic real-time behaviour, low power.

2.2 Register file — why 32 general-purpose registers?

RISC requires arithmetic operations on registers, not memory. Having 32 GPRs (R0–R31) means the CPU rarely fetches from SRAM during arithmetic. This is what enables single-cycle execution and 16 MIPS at 16 MHz.

Register	Purpose
R0–R25	General purpose — arithmetic, logic, data
R26:R27 (X)	X pointer — indirect memory addressing
R28:R29 (Y)	Y pointer — frame pointer for local variables
R30:R31 (Z)	Z pointer — table lookups (LPM from Flash)
SREG	Status register: I T H S V N Z C flags

Key Point

SREG I-bit is critical. SREG bit 7 = I (Global Interrupt Enable). When I=1, interrupts are enabled. `sei()` sets it; `cli()` clears it. When an interrupt fires, hardware automatically clears I — so nested interrupts don't happen unless you explicitly call `sei()` inside the ISR (rare). The C flag is set when addition overflows 8 bits; the Z flag when a result is zero — both used in conditional branches.

2.3 RISC characteristics (AVR)

- Most instructions execute in 1 clock cycle — the defining RISC property.
- Fixed-length instructions (16 or 32 bits) — easy to fetch and decode.
- Load-store architecture: operations only on registers, not directly on memory.
- Large register file (32) reduces memory traffic.
- Simple pipelining possible because decode is predictable.

2.4 The 2-stage pipeline

AVR uses a 2-stage pipeline: while Instruction N is executing, Instruction $N + 1$ is being fetched from Flash. Effective throughput approaches 1 instruction per cycle.

Clock	Stage 1 (Execute)	Stage 2 (Fetch)
Clock 1	(nothing)	FETCH <code>add R16,R17</code>
Clock 2	EXECUTE <code>add R16,R17</code>	FETCH <code>mul R18,R19</code>
Clock 3	EXECUTE <code>mul R18,R19</code>	FETCH <code>next</code>

Critical / Pitfall

Pipeline flush. When a branch is *taken* (if/else, function call) or an interrupt fires, the pre-fetched instruction is wrong. The pipeline must be flushed — costs 1–4 clock cycles. This is why branches take 1 cycle if not taken but 2 cycles if taken (per AVR datasheet). Minimize branching inside time-critical ISRs.

2.5 Harvard vs von Neumann

Property	Harvard (AVR, most MCUs)	Von Neumann (x86, ARM-A)
Buses	Separate program + data buses	Single shared bus
Simultaneous access	Yes: fetch + data at same time	No: bottleneck
Code/data separation	Physical — can't confuse	Same address space
Typical use	MCUs, DSPs	PCs, application processors

Key Point

Why this matters in your exam. Harvard is WHY `PROGMEM` exists. In C on AVR, global constants are copied to SRAM at startup. A 256-byte sine table declared normally wastes 256 bytes of your 8 KB SRAM (31 %). Declare with `PROGMEM` and read with `pgm_read_byte()` to keep it in Flash.

```
const uint8_t sin_table[256] PROGMEM = {0, 3, 6, ...};
uint8_t val = pgm_read_byte(&sin_table[45]);
```

Chapter 3

Lecture 3: Memory System

Memory selection is the most common calculation question. Know the three types, when to use each, and how to calculate whether data fits.

3.1 Memory overview

Type	Size	Volatile?	Use for
Flash	128 KB	No	Program code, constants (PROGMEM)
SRAM	8 KB	Yes	Runtime variables, buffers, stack
EEPROM	4 KB	No	Settings, calibration (rarely written)

3.2 Memory decision table

Data type	Memory	Reason
Program code / algorithm	Flash	Code never changes at runtime
Sine/cosine lookup table	Flash (PROGMEM)	Constant; keep in Flash to save SRAM
Current sensor reading	SRAM	Volatile, frequently updated
Temporary calc result	SRAM (stack)	Exists only during function execution
PID gains / motor calibration	EEPROM	Non-volatile, rarely written
ZigBee channel	EEPROM	Set at installation, non-volatile
30-day rolling average	EEPROM	Updated daily ($\ll$ 100k limit)
5000 ADC samples (10 KB)	Stream via UART	10 KB > 8 KB SRAM
Long-term data log	External SPI flash/SD	Too large for on-chip memory

3.3 SRAM buffer sizing

Formula & Calculation

$$\text{Buffer (bytes)} = N_{\text{samples}} \times \text{bytes/sample}$$

Example: $200 \text{ Hz} \times 30 \text{ s} \times 5 \text{ bytes} = 30,000 \text{ bytes} > 8,192$ — cannot buffer. Stream via UART.
UART capacity: $115\,200 \text{ baud} = 11,520 \text{ bytes/s}$. Required: $200 \times 5 = 1000 \text{ bytes/s}$. ✓

3.4 SRAM memory map

Address	Region	Used for
0x0000–0x001F	Register file (R0–R31)	CPU general-purpose registers
0x0020–0x00FF	I/O registers	Peripheral control (TCCR1A, ADC-SRA, ...)
0x0100–0x20FF	Internal SRAM (8 KB)	Globals (from 0x0100) + Stack (from 0x20FF down)

Critical / Pitfall

Stack overflow — the silent killer. Stack grows *down* from 0x20FF; globals grow *up* from 0x0100. Large local arrays in functions, deep call chains, or ISRs that call many functions cause a collision that overwrites globals silently. Avoid large local arrays; prefer static/global buffers with known sizes.

What goes on the stack during an interrupt:

1. Hardware pushes PC (2 bytes), decrements SP by 2.
2. Hardware clears I-bit.
3. CPU jumps to ISR vector.
4. ISR saves SREG + used registers.
5. ISR runs. ISR restores registers + SREG.
6. RETI — pops PC, re-enables I-bit. Main resumes.

3.5 EEPROM endurance

Formula & Calculation

$$\text{Lifetime (years)} = \frac{100,000}{\text{writes/day} \times 365}$$

Every 60 s: 1440 writes/day $\Rightarrow$ 0.19 years. Too short — use external flash.

Daily update: 1 write/day $\Rightarrow$ 273 years. Fine.

Write time: $\approx$ 3.4 ms/byte. Never write EEPROM inside an ISR or fast loop.

Use `eeeprom_update_word()` (only writes if value changed) to extend life further.

3.6 Clocking system

Source	Accuracy	Use when
Crystal (8/16 MHz)	$\pm$ 20–50 ppm	Almost always: UART, USB, accurate timing
Internal RC	$\pm$ 10 %	Quick prototype only, no communications
32.768 kHz crystal	$\pm$ 5–20 ppm	RTC, ultra-low-power wakeup

Critical / Pitfall

Why $\pm$ 10 % destroys UART. At 115 200 baud, each bit is 8.68 μ s. A 10 % clock error accumulates to 0.9 bit-period over 9 bits — the receiver samples the wrong bit. UART requires $<$ 2 % clock accuracy. **Always use a crystal for any communicating system.**

Chapter 4

Lecture 4: Advanced Hardware

4.1 ARM — why it exists

ARM is not a chip — it is an Instruction Set Architecture licensed to manufacturers (Atmel, STM, NXP, ...). ARM doesn't make chips; it designs the ISA and licenses it.

ARM's RISC philosophy gives low transistor count — low power at equivalent performance. This is why ARM dominates mobile and IoT.

Core	Perf.	Key feature	Choose for
Cortex-M0+	52 DMIPS	Lowest power	Simple IoT, replacing 8-bit MCUs
Cortex-M3	125 DMIPS	Hardware divide	General embedded control
Cortex-M4	180 DMIPS	DSP + FPU	Motor drives, audio, signal processing
Cortex-M7	300+ DMIPS	Dual-issue	High-performance real-time
Cortex-A5	850 DMIPS	MMU — runs Linux	IoT gateway, Raspberry Pi-class

Key Point

Choose ARM Cortex-M over AT90USB when: (1) Floating-point required (PID with float, FFT). (2) Need >128 KB Flash or >8 KB RAM. (3) Need faster comms (USB full-speed, Ethernet). (4) Control loop too tight for 16 MHz. But: ARM needs more complex init, higher power, higher cost. Don't upgrade just because it's more powerful.

4.2 MCU vs MPU

Requirement	MCU ok?	Why
Run full Linux OS	No → MPU	MCU has no MMU
Complex graphical touch-screen UI	No → MPU	Needs GPU, GB RAM
Gigabit Ethernet / USB host	No → MPU	Hardware too complex for 8-bit
More than 2 MB RAM	No → MPU	MCU SRAM is KB-range
< 1 μ s interrupt response	Yes	OS jitter rules out MPU
Battery powered, years of operation	Yes	MPU power too high
Motor control, sensors, simple comms	Yes	MCU sufficient, simpler, cheaper

4.3 DMA and event systems

Key Point

Problem without DMA: Reading 1000 ADC samples requires $1000 \times (\text{read, store, increment, check}) = \text{thousands of CPU cycles}$ wasted on data movement.

Solution: DMA controller transfers autonomously. CPU sets source, destination, count — DMA fills the buffer while CPU runs the control algorithm.

Disk project: DMA transfers ADC results directly to SRAM on each conversion complete. CPU free to run PID.

Event system (XMEGA): Peripheral-to-peripheral connections bypass the CPU entirely. Timer overflow → ADC start → DMA moves result. CPU only wakes to process a full buffer.

Chapter 5

Lecture 5: Timers, ADC, PWM, Interrupts, Communications

5.1 Timers — the backbone of real-time

Mode	How it works	Use for
Normal	Counts 0→TOP, overflow interrupt	Coarse timing
CTC	Counts 0→OCR, resets, ISR at OCR	Precise periodic interrupts
Fast PWM	Counts 0→TOP, OCR sets duty	Motor/LED PWM
Input Capture	Edge on ICP1 → timer value copied to ICR1	Slot event timestamping

Formula & Calculation

CTC mode — the most common exam formula:

$$\text{OCR} = \frac{f_{cpu}}{\text{prescaler} \times f_{desired}} - 1$$

Example: 200 Hz at 16 MHz, prescaler 64: $\text{OCR1A} = 16,000,000 / (64 \times 200) - 1 = 1249$. Verify: $f = 16,000,000 / (64 \times 1250) = 200 \text{ Hz}$. ✓

Input Capture timing:

$$\Delta t = \frac{\text{ICR1}_{new} - \text{ICR1}_{old}}{f_{timer}} \quad T_{rev} = \frac{60}{RPM} \quad T_{slot} = T_{rev} \times f_{slot}$$

Example: 1000 RPM, 1/8 slot fraction: $T_{slot} = (60/1000)/8 = 7.5 \text{ ms}$.

5.2 ADC — sensing the physical world

AT90USB1287 Specs — Know These

Resolution: 10-bit ($2^{10} = 1024$ steps) Channels: 8 (mux) Reference: AVCC, AREF pin, or internal 2.56 V

ADC clock prescalers: 2, 4, 8, 16, 32, 64, 128 Recommended: 50–200 kHz for max accuracy

Conversion time: 13 ADC clock cycles Max sample rate: $\approx 9600 \text{ Hz}$ at 200 kHz ADC clock

Formula & Calculation

$$\text{Step size} = \frac{V_{ref}}{2^n - 1} \quad (5\text{ V, 10-bit : } 4.88\text{ mV/step})$$

$$V_{measured} = ADC \times \frac{V_{ref}}{1023} \quad RPM = ADC \times \frac{RPM_{max}}{1023}$$

Minimum sample rate:

$$f_{min} = \frac{RPM_{max}}{60} \times N_{samples/rev}$$

Example: 1000 RPM, 10 samples/rev: $f_{min} = (1000/60) \times 10 = 166.7\text{ Hz} \rightarrow$ use 200 Hz.
 ADC clock for 200 Hz: $200 \times 13 = 2600\text{ Hz min.}$ Prescaler 128: 125 kHz. ✓

Critical / Pitfall

If the internal 2.56 V reference is selected in software while a 5 V source is connected to the AREF pin, an internal short circuit can permanently damage the MCU analogue circuitry.

5.3 PWM — controlling actuators

PWM switches a digital pin rapidly between HIGH and LOW. The ratio of HIGH time to total period = duty cycle. The average output voltage = $V_{supply} \times DC$.

Formula & Calculation**Fast PWM, 8-bit (TOP = 255):**

$$f_{PWM} = \frac{f_{cpu}}{\text{prescaler} \times 256} \quad DC = \frac{OCR}{255} \quad V_{avg} = V_{supply} \times DC$$

Example: 16 MHz, prescaler 8: $f_{PWM} = 16,000,000 / (8 \times 256) = 7812.5\text{ Hz.}$

For DC motors: use $f_{PWM} \gg$ mechanical time constant (typically $>1\text{ kHz}$). For LED dimming: $>50\text{ Hz}$.

PID control with PWM:

1. Read ADC (tachogenerator) $\rightarrow$ convert to RPM.
2. Error: $e = \text{setpoint} - \text{measured}$.
3. PID: $u = K_p e + K_i \int e dt + K_d (de/dt)$.
4. Clamp $u \in [0, 255]$.
5. Write to OCR — sets PWM duty cycle.

K_p alone: reduces error but leaves steady-state offset. $+K_i$: eliminates offset. $+K_d$: damps overshoot.

5.4 Interrupts — real-time response

Use interrupts when	Use polling when
Missing event causes data loss (UART byte)	Simple, slow I/O that can wait
Precise timing required (slot capture)	Interrupt overhead unacceptable
Event is infrequent, unpredictable	Event predictable and continuous
CPU should sleep between events	CPU always busy anyway

Key Point

Three steps to enable an interrupt:

1. Set the peripheral interrupt enable bit (e.g. OCIE1A for Timer1 CTC).
2. Set the global enable: call `sei()`.
3. The triggering event must actually occur.

ISR rules:

1. Short and fast: set a flag, copy a byte. Process in `main()`.
2. Never call blocking functions: no `printf`, no `_delay_ms`, no EEPROM writes.
3. Shared variables must be `volatile`.
4. Use `cli()/sei()` around multi-byte critical sections in `main()`.
5. AVR has no interrupt priorities — lower vector address wins.

5.5 UART / SPI / I²C — protocol comparison

Protocol	Wires	Speed	Devices	Best for
UART	2 (TX, RX)	≤1 Mbps	2 only	Debug, PC comms, ZigBee
SPI	3+1 CS/dev	10s MHz	Multi	SD card, flash, display
I ² C	2 (SDA, SCL)	100k–1 MHz	127	Multiple slow sensors

UART key facts: Asynchronous — both sides agree on baud rate. Frame: [START=0] [8 data bits] [STOP=1]. At 115 200 baud: 1 byte = 86.8 μs. Throughput: 11 520 bytes/s.

$$\text{UBRR} = \frac{f_{cpu}}{16 \times \text{baud}} - 1 \quad (16 \text{ MHz, } 115\,200 \text{ baud: } \text{UBRR} \approx 8)$$

U2X mode halves the divisor: $\text{UBRR} = 16$, error = 0.16 %. Use U2X for accuracy.

SPI key facts: Synchronous, full-duplex. Master drives SCK and CS (active low). MOSI/MISO shared. 4 modes (CPOL × CPHA) — must match device datasheet. Fastest protocol (4–8 MHz typical).

I²C key facts: Synchronous, half-duplex, open-drain. Needs pull-up resistors (4.7 kΩ to VCC). Transaction: START → 7-bit address + R/Ḡ → ACK → data → STOP. Up to 127 devices on 2 wires. Vineyard: temp (0x48), humidity (0x5F), RTC (0x68) all on one bus.

Key Point

Single-ended vs differential signalling. Single-ended (UART, SPI, I²C): signal referenced to ground — noise couples directly. Differential (USB, Ethernet, CAN, RS-485): two complementary wires; common-mode noise cancels. Use differential for anything >1 m or in electrically noisy environments (motors).

5.6 USB and Ethernet

USB class	Speed	Use case
HID	1.5 Mbps	Keyboard/mouse
CDC (Virtual Serial)	12 Mbps	Appears as COM port; use <code>printf()</code> normally
Mass Storage	12 Mbps	Appears as USB drive
High Speed (USB 2.0)	480 Mbps	Data acquisition — needs dedicated hardware

USB CDC — the practical solution for data logging. The AT90USB1287 has built-in USB hardware. Using the Atmel CDC library, the MCU appears as a COM port. Data travels at 12 Mbps instead of 115 200 bps: 30 KB of data in 30 ms vs 2.6 s over UART.

Ethernet — why 8-bit MCUs struggle. 100 Mbps = 12.5 MB/s incoming data. AT90USB at 16 MHz cannot process this. TCP/IP stack needs 4–16 KB RAM for buffers. Solutions: (1) SPI Ethernet controller (ENC28J60) handles hardware, MCU processes complete frames. (2) ARM MCU with hardware Ethernet MAC. (3) MPU running Linux.

Chapter 6

Lecture 6: IoT Architecture and Power Management

6.1 IoT hierarchy

Layer	Term	Hardware example	Role
Thing (sensor)	SS	AT90USB + tachogenerator	Measure, local processing
Thing (actuator)	SA	AT90USB + motor driver	Control physical world
Edge computing	LCR	Raspberry Pi, gateway	Aggregate, local analytics
Internet	GC	4G router, WiFi AP	Bridge local ↔ cloud
Cloud	RCR	AWS IoT, Azure	Long-term storage, ML, alerts

At the MCU: real-time sensing (10–200 Hz), local control loop, short-term history (seconds). At edge: aggregate, medium-term trends. At cloud: long-term storage, ML, global analytics — computationally too expensive for the MCU.

6.2 Power and sleep modes

Mode	Current	Wake source	Use for
Idle	≈5 mA	Any interrupt	Fast-response waiting
ADC Noise Reduction	≈1 mA	ADC complete	Quieter ADC conversions
Power-down	< 1 μA	Ext INT, WDT, TWI	Max battery life (IoT default)
Power-save	< 1 μA	Timer2 async, ext INT	Power-down + Timer2 RTC
Standby	≈20 μA	Ext INT, WDT	Fast wake, moderate saving

Formula & Calculation

Average current and battery life:

$$I_{avg} = \frac{I_{active} \times t_{active} + I_{sleep} \times t_{sleep}}{T_{cycle}} \quad \text{Life (h)} = \frac{C_{mAh}}{I_{avg}}$$

Vineyard example (60 s cycle, 500 mAh Li-Po):

Active 2 s: MCU 10 mA + ZigBee TX 205 mA + sensors 0.4 mA = 215.4 mA. Sleep 58 s: ≈ 0.002 mA total.

$$I_{avg} = \frac{215.4 \times 2 + 0.002 \times 58}{60} = 7.18 \text{ mA} \Rightarrow \text{Life} = \frac{500}{7.18} = 2.9 \text{ days}$$

Improve to 30 days: Sense every 10 min instead of 1 min: $I_{avg} = 0.72 \text{ mA} \Rightarrow \text{Life} = 694 \text{ h} = 29 \text{ days}$.

Key Point

Dominant IoT power consumer: Wireless TX (ZigBee $\approx 205 \text{ mA}$) is $200,000\times$ more than MCU Power-down ($< 1 \mu\text{A}$). Keep wireless TX time as short as possible. Compress data before transmitting. Use the lowest TX power level that achieves range.

Chapter 7

Lecture 7: Hardware Design Process

7.1 The process order — why I/O first?

You cannot choose an MCU until you know what peripherals you need. Choosing MCU first risks selecting one without enough ADC channels or I²C interfaces.

Design order:

1. Draw a block diagram: MCU centre; all I/O devices with interface labels; power supply; JTAG/ISP.
2. Select I/O devices: output type, package, voltage, accuracy, current, cost.
3. Tally required MCU I/O (# digital I/O, ADC channels, UARTs, SPIs, I²C, PWM, ISP).
4. Select MCU (filter by peripherals, then memory, clock, voltage, pin count, power modes, cost).
5. Design power supply (sum current per rail; add 20 % margin; choose regulator type).

7.2 I/O device categories

Category	Examples	Typical interface
Sensors	Temperature, humidity, RPM, pressure	ADC (analog), I ² C, SPI
Actuators	DC motor, servo, solenoid	PWM, GPIO (via driver circuit)
User input	Button, keypad, rotary encoder	GPIO (with debounce), interrupt
Display	LCD, OLED, LED	Parallel, SPI, I ² C, GPIO
Communication	ZigBee module, GPS, Bluetooth	UART (most common)
Storage	SD card, SPI flash, EEPROM	SPI, I ² C

7.3 Power supply: switching vs linear

Type	Pros	Cons	Use when
Linear (LDO)	Low noise, simple	Inefficient ($V_{in} \gg V_{out}$)	Small ΔV , noise-sensitive
Switching (buck)	>80 % efficient	EMI, complex	Large ΔV , efficiency critical
Buck → LDO	Efficient + clean	Two stages	Battery: switch down, LDO filters

7.4 MCU selection criteria (vineyard example)

- PicoPower mode: sub-microamp sleep current — critical for battery life.
- 1.8–5 V operation: compatible with Li-Po voltage range.

- All required peripherals: $1 \times$ UART, $1 \times$ SPI, $1 \times$ I²C, $3 \times$ ADC, $5 \times$ GPIO.
- 32 KB Flash: sufficient for sensor drivers + control + ZigBee protocol.
- 8-bit AVR: existing tools, library support.

Chapter 8

Lecture 8: Software Engineering

8.1 Four process activities

Activity	What it means
Specification	Customers + engineers define what the software must do and its constraints. Requirements are written <i>before</i> coding.
Development	Software is designed and programmed. Follows architectural design → modularization → coding.
Validation	Software is tested to verify it meets requirements (unit, integration, system testing).
Evolution	Software is modified as requirements change. Maintenance often exceeds development cost.

Key Point

Where do bugs come from? (Mogyorodi, 2003): Requirements: 56 %. Design: 27 %. Code: 7 %. Other: 10 %. Getting requirements right before coding is critical. Fixing a bug post-deployment costs 200× more than at requirements stage.

Cost pyramid: Requirements (1×) → Design (5×) → Development (10×) → Testing (20×) → Live (200×).

8.2 Functional vs non-functional requirements

Functional (WHAT)	Non-Functional (HOW WELL)
Measure temperature every 60 s	Accuracy: $\pm 0.5^{\circ}\text{C}$
Transmit wirelessly to farmhouse	Wireless range: $> 1\text{ km}$
Store data if communication fails	Operate for 2 years on battery
Detect disk slot crossing	Slot detection latency: $< 1\text{ ms}$
Control motor speed via PWM	Must operate -10°C to 50°C

Key Point

Non-functional requirements drive hardware selection. “Battery life 2 years” → average current calculation. “Latency $< 1\text{ ms}$ ” → which timer mode, what clock. “ $\pm 0.5^{\circ}\text{C}$ accuracy” → which

sensor, what ADC resolution.

8.3 The project report checklist (Dr. Al-Anbuky's 11 points)

1. Functional and non-functional requirements
2. Block diagram of the system
3. Commercial components for each major block (with specs)
4. Relevant API/library components for the lab board
5. Key external and internal events
6. Required use of microcontroller timers
7. ADC and/or PWM module usage
8. Communication protocol(s) and approach
9. Data memory requirements
10. Power and energy requirements
11. State diagram showing operational flow

8.4 State machine modelling

A Finite State Machine (FSM) divides a program into states. Transitions are triggered by events. Each state has associated actions.

Disk synchronisation FSM: IDLE (motor off, awaiting start) → RAMP_UP (increasing PWM, reading ADC) → STEADY (PID active, Input Capture logging) → STEP_CHANGE (new setpoint, recording response) → STOP. Any state → STOP on fault or stop command.

Vineyard IoT FSM: DEEP_SLEEP ($< 1\mu\text{A}$, WDT wakes) → SENSE (power sensors, I²C read) → PROCESS (compute averages, update EEPROM) → TRANSMIT (ZigBee send, await ACK) → ERROR (if TX fails, store locally, retry) → DEEP_SLEEP.

8.5 Software testing

- **Validation testing:** Does the software do what the spec says? (Temperature reading falls in expected range.)
- **Defect testing:** Deliberately try to break it — extreme inputs, edge cases, out-of-range values.
- Testing can reveal the *presence* of bugs; never their *absence*.

Chapter 9

Lecture 9: Software Architectures and Defensive Design

9.1 Architecture patterns

Architecture 1 — Polling loop. One infinite loop checking all inputs.

```
while(1) {  
    if (sensor_ready) read_sensor();  
    update_display();  
    _delay_ms(100); // wastes CPU cycles  
}
```

Pros: simple. Cons: blocks CPU; misses fast events; not power-efficient.

Architecture 2 — Event-driven (interrupt-based). Main sleeps; ISRs set flags.

```
volatile uint8_t data_ready = 0;  
while(1) {  
    if (data_ready) { data_ready = 0; process_data(); }  
    else sleep_cpu(); // sleep until next interrupt  
}  
ISR(TIMER1_COMPA_vect) { ADCSRA |= (1<<ADSC); }  
ISR(ADC_vect)          { buffer[idx++] = ADC; data_ready = 1; }
```

Pros: CPU sleeps, power-efficient, responsive. Cons: race conditions — protect shared data with cli()/sei().

Architecture 3 — State machine (FSM).

```
typedef enum { IDLE, RAMP_UP, STEADY, STOP } state_t;  
state_t state = IDLE;  
while(1) {  
    switch(state) {  
        case IDLE:    do_idle();    break;  
        case RAMP_UP: do_ramp();    break;  
        case STEADY:  do_pid();     break;  
        case STOP:    do_stop();    break;  
    }  
}
```

Pros: clear structure, easy to extend, excellent for operational modes.

Architecture 4 — Operating System. Manages multiple tasks with time-slicing. Standard OS (Linux, Windows) does NOT guarantee timing — scheduler introduces jitter. Cannot be used for hard real-time.

Architecture 5 — RTOS. Designed to meet timing deadlines. Soft real-time: preferred but not critical. Hard real-time: all timing constraints **MUST** be met. Examples: FreeRTOS, RTLinux, SYS/BIOS.

Key Point

Best for disk/vineyard: Event-driven + FSM combined. FSM defines states; interrupts handle time-critical events; main loop checks flags and manages transitions; CPU sleeps in IDLE. Gives: clean structure + real-time response + power efficiency.

9.2 Defensive software design

Embedded systems must recover automatically from faults without operator intervention.

Technique	Purpose and implementation
Watchdog Timer	If normal code hangs, WDT resets MCU. Call <code>wdt_reset()</code> periodically. Essential in all safety-critical systems.
POST	Check all components at startup: SRAM, EEPROM checksum, sensor comms. Catches hardware faults before operation.
Checksums	Compute checksum of EEPROM settings on write; verify on read. Mismatch → load defaults.
Boundary checks	<code>if(rpm > 1100) rpm = 1000;</code> Before using any value, verify it is in a sensible range.
Re-write registers	Periodically rewrite critical register values in main loop to restore any EMI-corrupted registers.
<code>cli()/sei()</code>	Disable interrupts when updating multi-byte registers or shared variables.
Fill unused vectors	Define all unused interrupt vectors to a safe handler.

Critical / Pitfall

Software runaway. A power spike can corrupt the program counter, causing the MCU to execute code from a random Flash address — driving motors to full power or locking up permanently. With a watchdog: the WDT reset call stops executing → WDT expires → MCU resets → system recovers. Every safety-critical embedded system must have a watchdog timer.

9.3 Hardware/software partitioning

Function	Hardware approach	Software approach	Choice
UART/SPI/I ² C	Built-in peripheral	Bit-bang GPIO	Always hardware
Switch debouncing	RC + Schmitt trigger	Software timer	Software (cheaper PCB)
PWM generation	Timer peripheral	Toggle loop	Always hardware
Precise event timing	Input Capture	Read timer in ISR	Input Capture

Key Point

Use built-in hardware wherever available (DMA, Event System, Timer Input Capture, hardware CRC). Hardware peripherals run independently of the CPU and don't consume MIPS. Software implementations block the CPU.

Chapter 10

Lecture 10: Modular Software Design

10.1 The three-level hierarchy

1. **Modules** (3–5 per project): abstract groupings of related functionality. Disk project: Sensor, Control, Actuator, Event (capture), Communications.
2. **Source files** (1–2 per module): `.c/.h` pairs. Encapsulation: anything not in the header is private to the source file.
3. **Functions** (3–6 per file): each does one small, testable task. Never copy-paste code — write a function.

Advantages:

- **Develop:** team members work on different modules without conflicts.
- **Debug:** narrow the problem — wrong sensor readings → look only in Sensor Module.
- **Test:** smaller functions are easier to unit-test.
- **Reuse:** a well-written SPI driver can be used in a future project unchanged.

10.2 Driver level architecture

Level	Example	Responsibility
Low-level	<code>SPDR = data;</code> <code>while(!(SPSR&(1<<SPIF)));</code>	Direct register access; MCU-specific
Mid-level	<code>uint8_t</code> <code>spi_transfer(uint8_t</code> <code>data)</code>	Send/receive bytes; portable
High-level	<code>uint8_t readSPIReg(uint8,</code> <code>uint8)</code>	Device-specific user API

Wrapper functions simplify calling complex functions by providing a cleaner interface: setting default values, converting argument types, or combining several calls. Common when using third-party libraries.

Key Point

Code reuse strategy: Separate the hardware-specific part into a small, clearly labelled function. Keep the rest hardware-agnostic. Use `#ifdef` for different hardware targets. The same ADC driver can work on ATmega328P, AT90USB1287, or ATSAM21 with only the low-level register writes changed.

Chapter 11

Fully Worked Examples

Worked Example — Q1 — Disk speed sensing: 10 samples/rev at 1000 RPM max

Step 1 — Sample rate: $(1000/60) \times 10 = 166.7 \text{ Hz} \rightarrow$ use 200 Hz.

Step 2 — ADC: 10-bit, AVCC 5 V ref. Resolution = $5/1023 = 4.88 \text{ mV/step} = 0.98 \text{ RPM/step}$. ✓

Step 3 — Timer CTC: $\text{OCR1A} = 16,000,000/(64 \times 200) - 1 = 1249$. Timer1 CTC fires at 200 Hz, triggers ADC.

Step 4 — ADC clock: Prescaler 128 $\rightarrow f_{\text{ADC}} = 125 \text{ kHz}$. Conversion time = $104 \mu\text{s} \ll 5 \text{ ms}$ period. ✓

Step 5 — Memory: $200 \times 30 \times 2 = 12,000 \text{ bytes} > 8,192$. Cannot buffer $\rightarrow$ stream via UART.

Step 6 — UART: 11 520 bytes/s capacity. Required: 400 bytes/s. ✓

Worked Example — Q2 — Vineyard comms: protocol justification

Temperature + Humidity + RTC $\rightarrow$ I²C. All three are I²C-compatible (HTS221 at 0x5F, DS3231 at 0x68). I²C uses 2 wires regardless of device count; SPI needs 3 separate chip-select lines. Speed at 100 kHz is ample for sensors updating at $< 1 \text{ Hz}$. Open-drain bus requires $4.7 \text{ k}\Omega$ pull-ups (0.7 mA); GPIO-controlled pull-ups can be deactivated during sleep.

ZigBee $\rightarrow$ UART. XBee presents a serial UART interface. UART is point-to-point, exactly matching MCU $\leftrightarrow$ module. MCU sends ASCII frames; module handles IEEE-802.15.4 MAC internally. ZigBee TX $\approx 205 \text{ mA}$ — sleep between transmissions is essential.

Worked Example — Q3 — Disk sync: functional and non-functional requirements

Functional (WHAT):

1. Measure disk speed (0–1000 RPM) continuously.
2. Control motor speed via PWM to achieve target RPM.
3. Detect disk slot crossing events.
4. Log speed, PWM, and timestamp during step-response tests.
5. Transmit data to a PC.
6. Apply step change in target speed on operator command.

Non-Functional (HOW WELL):

1. Sampling rate $\geq 10 \text{ samples/rev}$ at 1000 RPM ($\approx 167 \text{ Hz}$).
2. Slot detection: $< 10 \mu\text{s}$ (hardware Input Capture required).
3. Control loop: $\geq 200 \text{ Hz}$ for stable PID.
4. Real-time streaming (30 s data = 12 KB $>$ SRAM).

5. PID gains survive power-off (EEPROM storage).
6. Motor stops safely if watchdog expires.

Worked Example — Q4 — Vineyard battery life calculation

Active: MCU 10 mA + ZigBee 205 mA + sensors 0.4 mA = 215.4 mA for 2 s. Sleep: 0.002 mA for 58 s.

$$I_{avg} = \frac{215.4 \times 2 + 0.002 \times 58}{60} = 7.18 \text{ mA} \quad \Rightarrow \quad \text{Life} = \frac{500}{7.18} = 69.6 \text{ h} = 2.9 \text{ days}$$

To reach 2-year life (17 520 h): $I_{avg, req} = 500/17,520 = 0.0285 \text{ mA}$. Active budget per minute: $0.0285 \times 60 - 0.002 \times 58 = 1.59 \text{ mAs} = 7.4 \text{ ms}$. Conclusion: 2 years requires <8 ms active per minute — impractical; needs solar charging or larger battery.

Worked Example — Q5 — Disk project: software architecture and modules

Architecture: Event-driven + FSM. **Modules:** (1) `adc.c/h` — speed sensing; (2) `pid.c/h` — PID algorithm; (3) `motor.c/h` — Timer1 Fast PWM; (4) `capture.c/h` — Input Capture; (5) `uart.c/h` — data streaming. **FSM:** IDLE → RAMP_UP → STEADY → STEP_TEST → STOP. **ISRs:** Timer1 CTC (200 Hz) triggers ADC; ADC complete stores to buffer; Input Capture saves ICR1. **EEPROM:** K_p, K_i, K_d stored and reloaded on reset.

Quick Reference — All Key Formulas

Timing and frequency

$$T_{clk} = \frac{1}{f_{cpu}} = 62.5 \text{ ns at } 16 \text{ MHz}$$

$$OCR = \frac{f_{cpu}}{\text{prescaler} \times f_{desired}} - 1$$

200 Hz, /64: OCR1A = 1249

$$f_{PWM} = \frac{f_{cpu}}{\text{prescaler} \times 256} \quad (8\text{-bit Fast PWM})$$

$$T_{rev} = \frac{60}{RPM} \quad T_{slot} = T_{rev} \times f_{slot}$$

ADC

$$\text{Resolution} = \frac{V_{ref}}{1023} \approx 4.88 \text{ mV at } 5 \text{ V}$$

$$V = ADC \times \frac{V_{ref}}{1023} \quad f_{min} = \frac{RPM_{max}}{60} \times N$$

UART

$$UBRR = \frac{f_{cpu}}{16 \times \text{baud}} - 1$$

$$\text{Throughput} = \frac{\text{baud}}{10} \text{ bytes/s}$$

Memory

$$\text{Buffer} = N \times \text{bytes/sample}$$

If > 8192 bytes → stream via UART.

$$\text{EEPROM life} = \frac{100,000}{\text{wr/day} \times 365} \text{ (yr)}$$

Power and battery

$$I_{avg} = \frac{I_{act}t_{act} + I_{slp}t_{slp}}{T}$$

$$\text{Life (h)} = \frac{C_{mAh}}{I_{avg}}$$

PWM and PID

$$DC = \frac{OCR}{255} \quad V_{avg} = V_{supply} \times DC$$

$$u = K_p e + K_i \sum e + K_d \Delta e$$

AT90USB1287 key specs

Architecture	8-bit AVR RISC, Harvard, 16 MHz
Flash	128 KB (100k endurance)
SRAM	8 KB; stack from 0x20FF downward
EEPROM	4 KB; 100k cycles; 3.4 ms/write
ADC	10-bit, 8 channels, max ≈9.6 kHz
Timers	T0/T2 (8-bit), T1/T3 (16-bit)
Sleep (Power-down)	< 1 μA; wake 1–16 ms
Active	≈10–20 mA at 16 MHz

System design checklist

- ☐ Identify physical quantities
- ☐ Trace full data flow
- ☐ Calculate sample rate
- ☐ Check SRAM vs buffer size
- ☐ Select + justify protocol
- ☐ Calculate Timer OCR
- ☐ Interrupt vs polling decision
- ☐ Calculate battery life
- ☐ List F + NF requirements
- ☐ Draw state machine
- ☐ List modules + source files
- ☐ MCU vs MPU decision